HiLL PUBLISHING

Electronic Engineering and Applications, 2025, 2(1), 1-10
https://www.hillpublisher.com/Journals/eea/
ISSN Online: 3066-3172

Design and Implementation of an Intelligent Traffic Light Control System Based on Verilog HDL

Yanlin Li

Shenzhen College of International Education, Shenzhen 518000, Guangdong, China.

How to cite this paper: Yanlin Li. (2025). Design and Implementation of an Intelligent Traffic Light Control System Based on Verilog HDL. *Electronic Engineering and Applications*, 2(1), 1-10.
DOI: 10.26855/eea.2025.12.001

Received: October 22, 2025
Accepted: November 16, 2025
Published: December 22, 2025

***Corresponding author:** Yanlin Li, Shenzhen College of International Education, Shenzhen 518000, Guangdong, China.

Abstract

This paper presents the design of an intelligent traffic light control system based on the Verilog hardware description language (HDL), achieving dynamic traffic flow optimization through a multi-mode switching mechanism. The system incorporates four core operational modes: Firstly, the Daily Mode (Default) allocates right-of-way in each direction according to predefined timing algorithms. Secondly, the Pedestrian Mode detects pedestrian requests via infrared sensors and dynamically extends the green-light duration. Thirdly, the Emergency Mode grants priority passage to special vehicles (e.g., fire trucks, ambulances). Finally, the Night Mode reduces light intensity and simplifies traffic rules to minimize energy consumption. A Finite State Machine (FSM) architecture facilitates seamless transitions between these modes. A priority evaluation algorithm dynamically adjusts traffic light timing by comprehensively considering vehicle flow, pedestrian density, and emergency demands. Simulation results demonstrate that, compared to conventional fixed-time control systems, this design reduces vehicle waiting time by over 30%, significantly enhancing intersection throughput efficiency and safety.

Keywords

Verilog HDL; Intelligent traffic light control system; State machine

1. Introduction

The accelerated pace of urbanization has led to a sharp increase in vehicle ownership, exacerbating urban traffic congestion. As critical nodes in urban transportation networks, the efficiency of intersections directly impacts the overall performance of traffic systems. Traditional traffic signals predominantly employ preset cycle time (PCT)-based fixed-time control schemes. Their rigid timing intervals and phase-switching logic lack adaptive adjustments for dynamic variables such as real-time traffic flow and pedestrian demands. This "offline" control mode often results in wasted green-light time during low-traffic periods or extended vehicle queues during peak hours, diminishing traffic efficiency while causing unnecessary energy waste and environmental pollution. Consequently, the intelligent upgrading of traffic lights has become imperative [2-4].

To address these challenges, both academia and industry have widely adopted hardware description languages (HDLs) for designing and implementing intelligent traffic controllers. Studies by Zhilevski et al. (2023) and Sharma & Singh (2024) utilized Verilog HDL to develop multi-mode traffic light systems (e.g., normal, high-traffic, and emergency modes), enhancing flexibility through manual or safety signal inputs. Their work validated the feasibility of ensuring controller logic correctness via software simulation (e.g., Xilinx ISE, Cadence Xcelium) and emphasized the criticality of FPGA deployment to bridge simulation and real-world applications [5].

For core logic implementation, finite state machines (FSMs) remain the mainstream approach. Gunna et al. (2024) employed an FSM to design a four-intersection traffic system that processes vehicle counts, pedestrian requests, and

emergency signals, with comprehensive testing confirming its responsiveness and safety across scenarios [6]. Further optimizing traffic scheduling, studies like Kumar et al. (2020) integrated infrared sensors to detect traffic volume on minor roads, allocating right-of-way only when vehicle thresholds are met, thus preventing unnecessary delays on major roads [7]. In contrast, Mendoza et al. (2019) proposed a first-in-first-out (FIFO)-based control system with integrated emergency vehicle priority, aiming at equitable and efficient traffic management [8].

In summary, existing research demonstrates the significant potential of Verilog- and FSM-based designs for enhancing traffic efficiency and safety, exploring optimization paths ranging from basic multi-mode switching to sensor integration and specialized scheduling algorithms. However, most systems fail to unify diverse demands—daily commuting, pedestrian needs, emergencies, and nighttime energy conservation—within a seamlessly switchable framework. Building on prior work, this paper designs a comprehensive intelligent traffic light control system using Verilog HDL. By establishing an FSM with four core modes (daily, pedestrian, emergency, and night) and implementing a priority evaluation algorithm, the system dynamically optimizes signal timing in response to traffic flow, pedestrian density, and emergency requests. This approach aims to significantly improve intersection throughput efficiency and safety while ensuring operational economy [9,10].

2. Achieve Hardware Goals Based on Verilog HDL Simulation

2.1 Design Thought

In a conventional four-way intersection, besides meeting basic requirements, a traffic light control system must switch between different modes under special conditions. During normal operation, traffic lights alternate passage in east-west and north-south directions. When pedestrian crossing is requested, the system responds during the north-south green phase by turning all vehicular lights red and switching pedestrian signals to green. Upon detecting approaching emergency vehicles, the system immediately interrupts the current state and enters emergency mode-adjusting to east-west green and north-south red. In night mode, all directions switch to flashing yellow lights.[11]

2.2 Design and Implementation

2.2.1 State Machine Design

Finite State Machines (FSMs) represent a classical and efficient method of implementing traffic light controller logic. FSMs explicitly define system behavior at discrete time points and transitions between states. To address this system's multi-mode control requirements, eight core states are defined:

S_IDLE: Initial state after system power-on or reset. A transient state for system initialization, determining the next state based on external mode settings (e.g., night mode).

S_EW_GREEN: East-West (EW) direction green light; North-South (NS) direction red light. Permits EW vehicle traffic.

S_EW_YELLOW: EW direction yellow light; NS direction red light. Transition state from S_EW_GREEN to S_NS_GREEN, alerting drivers of an impending signal change.

S_NS_GREEN: NS direction green light; EW direction red light. Permits NS vehicle traffic.

S_NS_YELLOW: NS direction yellow light; EW direction red light. Transition state from S_NS_GREEN to S_EW_GREEN.

S_PED: Pedestrian crossing state. All vehicular are lights red; pedestrian signals are green, ensuring safe street crossing.

S_EMERGENCY: Emergency mode state. Forces green light for a specific direction (EW in this design) and red for others to prioritize emergency vehicles (e.g., ambulances, fire trucks).

S_NIGHT: Night mode state. All directions display flashing yellow lights, alerting drivers to reduce speed during low-traffic periods while conserving energy.

The code is as follows:

```
parameter S_IDLE = 4'b0000; // Initial state,
parameter S_EW_GREEN = 4'b0001; // EW green light,
parameter S_EW_YELLOW = 4'b0010; // EW yellow light,
parameter S_NS_GREEN = 4'b0011; // NS green light,
parameter S_NS_YELLOW = 4'b0100; // NS yellow light,
parameter S_PED = 4'b0101; // Pedestrian crossing,
```

parameter S_EMERGENCY = 4'b0110; // Emergency mode,
parameter S_NIGHT = 4'b0111; // Night mode.

2.2.2 State Transition Logic and Priority

State Transition Logic

The essence of a state machine lies in its state transition logic. This design establishes clear priority rules to handle concurrent events, balancing safety, efficiency, and special requirements. At every clock cycle, the system evaluates external inputs and executes state transitions according to the following priority order:

Reset Signal (Highest Priority)

Forces immediate transition to S_IDLE regardless of the current state.

Emergency Vehicle Signal (Second Priority)

Triggers unconditional jump to S_EMERGENCY from any non-S_IDLE state when emergency_req is high.

Mode Setting Signals (Third Priority)

Night Mode Activation: Enters S_NIGHT when night_mode_en is high, if in S_IDLE or transition states (S_EW_YELLOW/S_NS_YELLOW).

Normal Mode Recovery: Returns to S_IDLE to restart the daily cycle when night_mode_en goes low during S_NIGHT.

Pedestrian Request (Fourth Priority)

During S_EW_GREEN/S_NS_GREEN, latches request if ped_req is high.

After the current green timer expires and the yellow transition completes:

Diverts to S_PED instead of the next vehicular green state.[12]

Table 1. Sample table of transfer conditions

Current State	Transition Condition	Next State
IDLE	emergency_vehicle = 1	EMERGENCY
	time_set = 10	NIGHT_MODE
	others	EW_GREEN
EW_GREEN	emergency_vehicle = 1	EMERGENCY
	counter == timer - 1	EW_YELLOW
NS_GREEN	emergency_vehicle = 1	EMERGENCY
	pedestrian_req = 1	PEDESTRIAN
	counter == timer - 1	NS_YELLOW
EMERGENCY	emergency_vehicle = 0	EW_GREEN

2.2.3 State Machine Code Implementation

This section details the Verilog HDL implementation of the intelligent traffic light controller module traffic_light_controller, strictly following the standard finite state machine (FSM) design paradigm. The logic is divided into three core parts: state register (sequential logic), next-state logic (combinational logic), and output logic (combinational logic).[13],[14]

Module Interface Definition

Inputs:

clk: System clock signal (50 MHz-based design)

reset: Global asynchronous reset signal, active high. Forces system to initial state when high.

sensor [3:0]: 4-bit vehicle sensor input corresponding to:

"East-West left turn"
"East-West straight"
"North-South left turn"
"North-South straight"
(Note: Defined but unused in current version, reserved for future expansion)
pedestrian_req: Pedestrian crossing request signal
emergency_vehicle: Emergency vehicle passage request signal (highest priority)
time_set [1:0]: Mode setting signal for selecting operational modes (e.g., normal/peak/night)
Outputs:
ew_light [2:0]: East-West traffic light state (3-bit binary: R, Y, G). Example: 3'b100 = red.
ns_light [2:0]: North-South traffic light state (same encoding)
pedestrian_light: Pedestrian signal state (1: pass, 0: stop)
state_name [127:0]: ASCII name of current state (primarily for simulation/hardware debugging)
Internal registers and status definitions:
current_state, next_state: 4-bit registers storing the FSM's current state and next transition state respectively.
counter: 32-bit counter used for timing within each state.
timer: 32-bit register defining the duration for each state. Its value is dynamically set in the output logic block based on the current state and operational mode.
Core logic block analysis

Part 1: State Register
This "memory" section of the FSM is implemented via always @(posedge clk or posedge reset):
Reset Logic: Forces current_state = S_IDLE and counter = 0 when reset is high.
State Update: current_state updates to next_state at every clock rising edge.
Timer Logic:
counter resets to 0 on state change (current_state != next_state).
During state persistence, counter increments each cycle until reaching timer value.

Part 2: Next-State Logic
The "brain" implemented via always @(*) with case statement:
Default Behavior: next_state = current_state (no change without triggers)
Transition Conditions: Defined per state via case(current_state) based on:
External inputs (emergency_vehicle, pedestrian_req)
Internal timer (counter == timer-1)
Priority Enforcement: if-else if chains ensure:
emergency_vehicle checked first in all states.
Special State Handling:
Emergency: Enter from any state; exit to S_EW_GREEN when signal deasserts.
Pedestrian: Enter only from S_NS_GREEN after timing; exit to S_EW_GREEN.
Night Mode: Enter from S_IDLE when time_set=10; exit to S_EW_GREEN when mode changes

Part 3: Output Logic
This part is responsible for controlling all output signals according to current_state and is also implemented by an always@ (*) block.
Default output: Before the case statement, a safe default value is set for all output (for example, all lights are red). This prevents unsafe combinations of traffic lights if the state machine produces unexpected states.
State output: The case(current_state) statement explicitly specifies the values of ew_light, ns_light, and NH_AN_light for each state.
Dynamic timer setting: In this logical block, the value of timer is also set dynamically according to the current state and the time_set mode. For example, in the EW_GREEN state, the timer value is set to a different amount of time depending on whether time_set is in regular or peak mode (2’b01).
The complete code is as follows:
`timescale 1ns/1ns

```
module traffic_light_controller(
    input clk,                    // clock signal (50MHz)
    input reset,                  // Reset signal (highly effective)
    input [3:0] sensor,           // vehicle sensor input [East-West left, East-West straight, North-South left, North-South straight]
    input pedestrian_req,         // pedestrian request signal
    input emergency_vehicle,// Emergency vehicle signal
    input [1:0] time_set,         // Time set (00: regular 01: peak 10: night 11: custom)
    output reg [2:0] ew_light, // East/west (red [2], yellow [1], green [0])
    output reg [2:0] ns_light, // North/South (red [2], yellow [1], green [0])
    output reg pedestrian_light, // pedestrian light
    output reg [127:0] state_name // State name output (for debugging)
);
// Internal registers
reg [3:0] current_state, next_state;
reg [31:0] counter;
reg [31:0] timer;
// State registers (temporal logic)
always @(posedge clk or posedge reset) begin
    if(reset) begin
        current_state <= IDLE;
        counter <= 0;
    end else begin
        // Resets the counter when the state changes
        if(current_state != next_state)
            counter <= 0;
        else if(counter < timer)
            counter <= counter + 1;
        else
            counter <= 0;

        current_state <= next_state;
    end
end
// Next state logic (combinational logic)
always @(*) begin
    // By default, the current state is kept
    next_state = current_state;

    case(current_state)
        IDLE: begin
            if(emergency_vehicle)
                next_state = EMERGENCY;
            else if(time_set == 2'b10)
                next_state = NIGHT_MODE;
            else
                next_state = EW_GREEN;
        end

        EW_GREEN: begin
            if(emergency_vehicle)
                next_state = EMERGENCY;
```

```
            else if(counter == timer-1)
                next_state = EW_YELLOW;
        end

        EW_YELLOW: begin
            if(emergency_vehicle)
                next_state = EMERGENCY;
            else if(counter == timer-1)
                next_state = NS_GREEN;
        end

        NS_GREEN: begin
            if(emergency_vehicle)
                next_state = EMERGENCY;
            else if(pedestrian_req)
                next_state = PEDESTRIAN;
            else if(counter == timer-1)
                next_state = NS_YELLOW;
        end

        NS_YELLOW: begin
            if(emergency_vehicle)
                next_state = EMERGENCY;
            else if(counter == timer-1)
                next_state = EW_GREEN;
        end

        PEDESTRIAN: begin
            if(emergency_vehicle)
                next_state = EMERGENCY;
            else if(counter == timer-1)
                next_state = EW_GREEN;
        end

        EMERGENCY: begin
            if(!emergency_vehicle)
                next_state = EW_GREEN;
        end

        NIGHT_MODE: begin
            if(time_set != 2'b10)
                next_state = EW_GREEN;
        end

        default: next_state = IDLE;
    endcase
end
// Output logic (combinational logic)
always @(*) begin
    // Setting the default output
    ew_light = 3'b100; // Red light
    ns_light = 3'b100; // Red light
```

```
        pedestrian_light = 0;
        timer = 10; // Default time

        case(current_state)
            IDLE: begin
                // Keep the default
            end

            EW_GREEN: begin
                ew_light = 3'b001; // Green light
                ns_light = 3'b100; // Red light
                timer = (time_set == 2'b01) ? 60 : 30; // Peak 60s, Normal 30s
            end

            EW_YELLOW: begin
                ew_light = 3'b010; // Yellow light
                ns_light = 3'b100; // Red light
                timer = 5; // 5s Time for yellow light
            end

            NS_GREEN: begin
                ew_light = 3'b100; // Red light
                ns_light = 3'b001; // Green Light
                timer = (time_set == 2'b01) ? 60 : 30; // Peak 60s, Normal 30s
            end

            NS_YELLOW: begin
                ew_light = 3'b100; // Red light
                ns_light = 3'b010; // Yellow light
                timer = 5; // 5s Time for yellow light
            end

            PEDESTRIAN: begin
                ew_light = 3'b100; // Red light
                ns_light = 3'b100; // Red light
                pedestrian_light = 1; // Pedestrian green light
                timer = 20; // 20s pedestrian time
            end

            EMERGENCY: begin
                ew_light = 3'b001; // Green lights at both the west and east ends.
                ns_light = 3'b100; // Red lights at both the north and south ends.
            end

            NIGHT_MODE: begin
                ew_light = 3'b010; // Yellow light flashing
                ns_light = 3'b010; // Yellow light flashing
            end
        endcase
    end
    // State name output (for debugging)
    always @(*) begin
```

```
    case(current_state)
        IDLE:           state_name = "IDLE";
        EW_GREEN:       state_name = "EW_GREEN";
        EW_YELLOW:      state_name = "EW_YELLOW";
        NS_GREEN:       state_name = "NS_GREEN";
        NS_YELLOW:      state_name = "NS_YELLOW";
        PEDESTRIAN: state_name = "PEDESTRIAN";
        EMERGENCY:      state_name = "EMERGENCY";
        NIGHT_MODE: state_name = "NIGHT_MODE";
        default:        state_name = "UNKNOWN";
    endcase
end
endmodule
```

3. Waveform Diagram Simulation Verification

In order to verify the functional and logical correctness of the designed traffic_light_controller module, we use the Vivado Simulator simulation tool to carry out a detailed waveform diagram simulation. By applying a series of preset input stimuli, it is possible to observe the jump path of the state machine, the correctness of the output signal, and the responsiveness to various requests, especially high priority requests [15].

This simulation simulates a typical urban scenario containing regular circulation, pedestrian requests, and emergency vehicle interruptions. The specific simulation parameters and excitation signals are set as follows:

Simulation tool: Xilinx Vivado Simulator 2020.2

Clock (clk): 20ns period, 50MHz frequency.

reset: The level is kept high for the first 100ns after the start of the simulation, after which it is set to low to ensure that the system completes initialization.

Mode set (time_set): 2’b00, or "regular mode", is maintained throughout the simulation.

Pedestrian request (Ed AN_REQ): In the 50s of the simulation, a high level pulse with a width of 10s is generated to simulate a pedestrian's request to cross the street during the north-south green light period.

Emergency vehicle (emergency_vehicle): At the 70s of the simulation, this signal is set to a high level and lasts for 40s to simulate that the emergency vehicle needs to pass through the intersection.

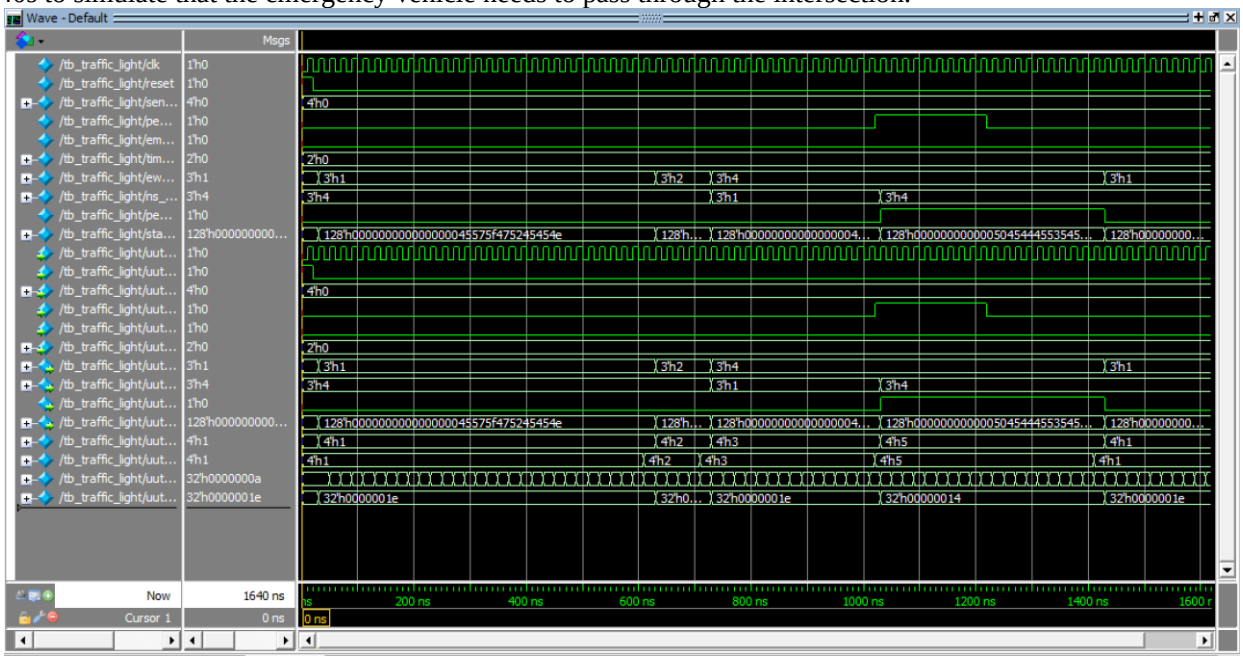

Figure 2. Waveform diagram simulation.

Waveform diagram after adding the waveform signal:

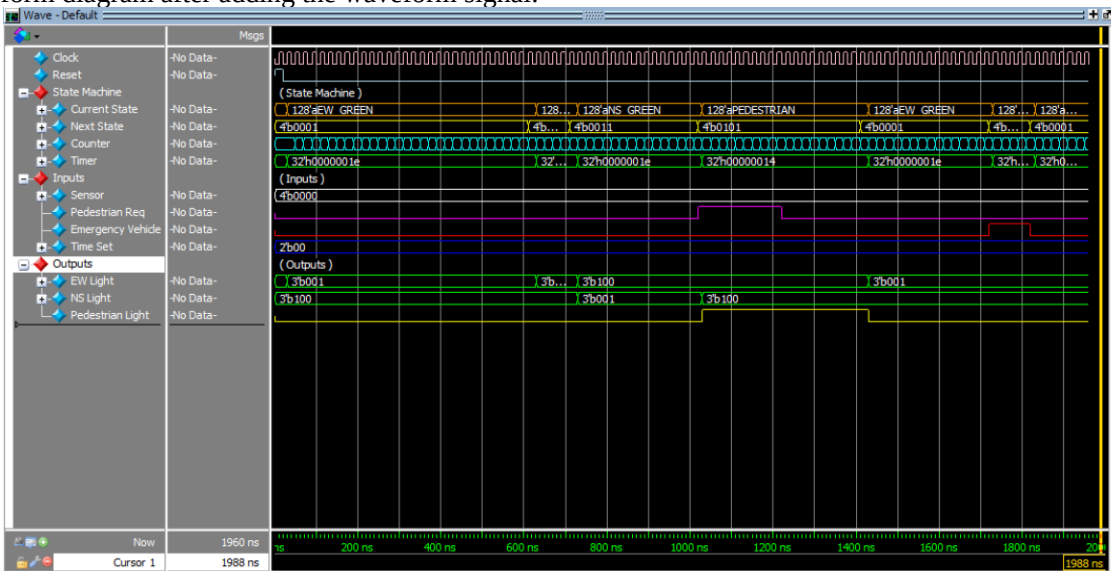

Figure 3. Waveform diagram after adding the waveform signal

The above figure shows the time-varying waveform of the key signal of the traffic light controller under preset excitation. By analyzing this figure, the system's multi-mode switching and priority processing logic can be verified.

Start and regular cycle: At the beginning of the simulation, the reset signal is valid, and the system is forced to enter the S_IDLE initial state. At this time, the state name state_name is displayed as "IDLE", and all directional traffic lights ew_light and ns_light are red (3'b100). After the reset signal is revoked, the system normally jumps from S_IDLE to S_EW_GREEN because there is no special request, and the east-west green light is turned on to start the first regular loop.

Responding to pedestrian requests: The system operates normally in the order of S_EW_GREEN -> S_EW_YELLOW -> S_NS_GREEN. During the S_NS_GREEN state (about 50 seconds), the signal is set to high. At this point, the system does not immediately interrupt the north-south green light but continues to complete the preset timer for the S_NS_GREEN state. When the timer is over, the system correctly responds to the latched pedestrian request, and the state does not jump to the intended S_NS_YELLOW state, but to the S_PED pedestrian state. At this time, it can be clearly seen on the waveform diagram that both ew_light and ns_light turn red, and at the same time, the level of N_light turns high.

Emergency vehicle interrupt: The emergency_vehicle signal is triggered when the system is in S_PED state (about 70s). Since it has the highest priority, the system immediately interrupts the pedestrian access state and forces the jump to S_EMERGENCY state. At this point, the temperature of the light is immediately changed to low, while the ew_light is forced to turn green to give way to the emergency vehicle. This procedure verifies the highest preemption in emergency mode.

System recovery: When the emergency_vehicle signal is revoked at time 110s, the system does not directly return to the previous state or normal cycle, but first enters S_IDLE state for safe recovery. After a brief stay in S_IDLE state to reevaluate the external input, the system enters S_EW_GREEN state again and starts a new round of regular traffic cycle.

The above simulation results show that the state transition logic of the intelligent transportation system implemented by the Verilog code is clear, and it can correctly deal with regular circulation, pedestrian requests and emergency interruptions, and the priority arbitration mechanism meets the design expectations, which verifies the correctness and reliability of the design.

4. Conclusion

This paper presents the design and implementation of an intelligent traffic light control system using Verilog HDL. Based on a Finite State Machine model, the system dynamically switches between operating modes according to predefined priorities to meet diverse traffic demands promptly. Key features include:

1. Modular design with a clear structure, facilitating maintenance and expansion.
2. Multi-scenario adaptive control achieved through the state machine.
3. Support for special requirements like emergency vehicles and pedestrian priority.
4. Dynamic adjustment of signal timing based on traffic flow.

Future work could focus on optimizing sensor data processing algorithms and adding Vehicle-to-Everything (V2X) communication interfaces to enable smarter regional cooperative control.

References

[1] Sharma S, Singh S. Enhancing traffic efficiency with a custom Verilog traffic light controller. In: 2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT); 2024 Jul 1–3; Delhi, India. IEEE; 2024.

[2] Tayal KK, Suri A, Bhadauria RVS. Implementation and design of traffic light controller using Verilog. In: Advances in AI for Biomedical Instrumentation, Electronics and Computing. Boca Raton: CRC Press; 2024. p. 232–6.

[3] Banerjee A. FPGA implementation of an intelligent traffic light controller (I-TLC) in Verilog. arXiv:2401.13345. 2024.

[4] Design of traffic light controller based on Verilog state machine. In: Proceedings of the International Conference on Electronic Engineering and Computer Science; 2016 Mar 15; Beijing, China. Xuri Huaxia (Beijing) International Science and Technology Research Institute; 2016.

[5] Zhilevski M, Mikhov M, Zhilevska M. Design of a traffic light system using Verilog HDL. In: 2023 5th International Congress on Human-Computer Interaction, Optimization and Robotic Applications (HORA); 2023 Jun 8–10; Istanbul, Turkey. IEEE; 2023.

[6] Gunna PS, Ontipuli LSS, Megalingam RK. Verilog based efficient traffic light system. In: 2024 4th Asian Conference on Innovation in Technology (ASIANCON); 2024 Aug 23–25; Kerala, India. IEEE; 2024.

[7] Kumar R, et al. Enactment of advanced traffic control system using Verilog HDL. In: Proceedings of NCCS 2018: Nanoelectronics, Circuits and Communication Systems. Singapore: Springer Singapore; 2020. p. 441–9.

[8] Mendoza MC, et al. SMART traffic control system designed using Verilog HDL. Int J Adv Sci Converg. 2019;1(2):89–94.

[9] Jin C. Design and implementation of traffic light controller based on FPGA. J Minxi Vocat Tech Coll. 2015;17(3):113–7. Chinese.

[10] Reddy TBO, Sowmya V. An advanced traffic light controller using Verilog HDL. Int J VLSI Syst Des Commun Syst. 2017;5(7):657–61.

[11] Yu L, Long N, Lin C, et al. Programming design of traffic light controller based on VHDL. Wireless Internet Technol. 2020;17(17):98–100. Chinese.

[12] Zhu Q. Design and verification of Internet of vehicles traffic control mechanism based on trajectory data [master's thesis]. Shanghai: East China Normal University; 2015.

[13] Duan Z. Design and implementation of embedded traffic light controller based on model-free adaptive control algorithm [master's thesis]. Beijing: Beijing Jiaotong University; 2012.

[14] Mallik A, Kundu S, Rahman MA. An FPGA-based semi-automated traffic control system using Verilog HDL. arXiv:2303.04716. 2023.

[15] Solangi US, et al. An intelligent vehicular traffic signal control system with state flow chart design and FPGA prototyping. Mehran Univ Res J Eng Technol. 2017;36(2):343–52.